

LABORATORY OBSERVATION / RECORD

COURSE NAME: FULL STACK WEB DEVELOPMENT

CODE: 23CM4121

Experiment No: 5

NAME: Reddy MOHINI

ROLL NO: A24126552110

TITLE OF THE EXPERIMENT

Implement Modules in Node.js

AIM

To implement and demonstrate the use of core (built-in / third-party) modules and user-defined ES modules in Node.js, and to develop a simple Express server using the Node.js module system

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
- To differentiate between CommonJS modules (require/module.exports) and ES modules (import/export).
- To use a core/third-party module (Express) with require() to build a server.
- To create user-defined ES modules and export functions from them.
- To import and use functions from user-defined modules in another file.

To test and verify the output of each module.

4. THEORY

A module in Node.js is a reusable block of code whose functionality can be exported and used by other files. Node.js supports two module systems:

- CommonJS Modules — use require() to import and module.exports (or exports) to export functionality. This is the default module system for .js files unless the project's package.json specifies "type": "module".
- ES Modules (ECMAScript Modules) — use the import and export keywords. Files use the .mjs extension, or a .js extension inside a project whose package.json contains "type": "module". Express is a third-party (core) module installed via npm and loaded using require('express'). It simplifies creating an HTTP server

and defining routes. In this experiment, app.js and coremodules.js use the CommonJS style (require) to load Express and start a server. greetings.mjs and Math.js are user-defined ES modules that export functions using the export keyword; these functions are imported and used in a separate driver file (index.mjs) using the import keyword.

The basic flow of the application is:

Module file (export) → Driver / Server file (require or import) → Node.js Processing → Console / Browser Output

5. ALGORITHM / PROCEDURE

- Create a new Node.js project folder and initialize it using npm init.
- Install Express using npm install express.
- Create app.js, import Express using require(), and define a GET route on '/'.
- Start the server with app.listen() on port 3000 and verify the output in a browser.
- Create coremodules.js following the same approach to reinforce module usage with require().
- Create a user-defined ES module greetings.mjs that exports sayHello() and sayGoodbye().
- Create a user-defined ES module Math.js that exports add() and subtract() using export { }.
- Add a package.json with "type": "module" inside the ES-modules folder so Math.js is treated as an ES module.
- Create a driver file (index.mjs) that imports the functions from greetings.mjs and Math.js.
- Call the imported functions and print the results to the console.
- Run each file using Node.js and observe the console/browser output.
- Verify the actual output against the expected output for each module

PROGRAM

app.js

```
const express = require('express');  
  
// Create Express application  
  
const app = express();  
  
// Define port  
  
const PORT = 3000;  
  
// Home route
```

```
app.get('/', (req, res) => {  
  res.send('Hello! Welcome to my Express server.');
```



```
});  
  
// Start server  
  
app.listen(PORT, () => {  
  console.log(`Server is running at http://localhost:${PORT}`);  
});
```

Coremodules.js

```
const express = require('express');
```



```
// Create an Express application  
  
const app = express();  
  
// Define the port  
  
const PORT = 3000;  
  
// Create a route for the home page  
  
app.get('/', (req, res) => {  
  res.send('Hello! Welcome to my Express server.');
```



```
});  
  
// Start the server  
  
app.listen(PORT, () => {  
  console.log(`Server is running at http://localhost:${PORT}`);  
});
```

greetings.mjs

```
export function sayHello() {  
  console.log('Hello');
```



```
}  
  
export function sayGoodbye()
```

```
console.log('Goodbye');  
}
```

Math.js

```
function add(a, b) {  
    return a + b;  
}  
  
function subtract(a, b) {  
    return a - b;  
}  
  
export { add, subtract };
```

index.mjs (driver file that uses the modules)

```
import { sayHello, sayGoodbye } from './greetings.mjs';  
  
import { add, subtract } from './Math.js';  
  
console.log('--- greetings.mjs module ---');  
  
sayHello();  
  
sayGoodbye();  
  
console.log('--- Math.js module ---');  
  
console.log('add(10, 5) =', add(10, 5));  
  
console.log('subtract(10, 5) =', subtract(10, 5));
```

7. CODE EXPLANATION

code	Explanation
require('express')	Imports the Express core module.
express()	Creates an Express application instance.
app.get('/')	Defines a GET route for the home page
app.listen()	Starts the server on the specified port.
Export function ..	Exposes a function from an ES module so other files can import it.
export { add, subtract }	Named export of multiple functions from Math.js

import { ... } from '...'	Imports named exports from another ES module
"type": "module"	package.json setting that tells Node.js to treat .js files as ES modules.
sayHello() / sayGoodbye()	User-defined functions that print greeting messages to the console.
add(a, b) / subtract(a, b)	User-defined functions that perform arithmetic and return the result.

8. TEST CASES AND EXECUTION DATA

Each file was executed individually using Node.js and the actual output was recorded against the expected output.

Test Case	File / Module	Action	Expected Result	Actual Result	status
TC-01	app.js	Run 'node app.js'	Server starts on port 3000	Server is running at http://localhost:3000	Pass
TC-02	app.js	Open http://localhost:3000/	Greeting message displayed	Hello! Welcome to my Express server.	Pass
TC-03	coremodules.js	Run 'node coremodules.js'	Server starts on port 3000	Server is running at http://localhost:3000	Pass
TC-04	coremodules.js	Open http://localhost:3000/	Greeting message displayed	Hello! Welcome to my Express server.	Pass
TC-05	greetings.mjs	Call sayHello() and sayGoodbye()	'Hello' and 'Goodbye' printed	Hello / Goodbye printed to console	Pass
TC-06	Math.js	Call add(10,5) and subtract(10,5)	Returns 15 and 5	add = 15, subtract = 5	Pass

Output:

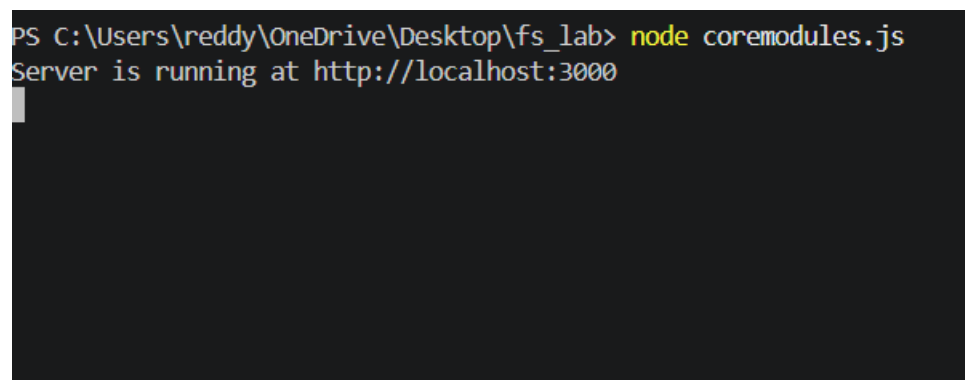
Output 1: app.js — Server Execution (Terminal)

```
PS C:\Users\reddy\OneDrive\Desktop\fs_lab> node app.js
Server is running at http://localhost:3000
```

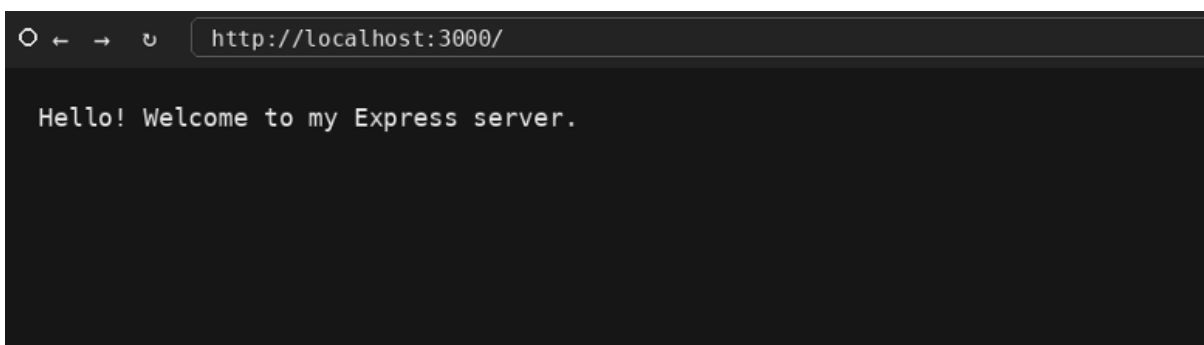
Output 2: app.js — Browser Output



Output 3: coremodules.js — Server Execution (Terminal)



Output 4: coremodules.js — Browser Output



Output 5: greetings.mjs and Math.js — via index.mjs (Terminal)

```
PS C:\Users\reddy\OneDrive\Desktop\fs_lab> node greeting.mjs
PS C:\Users\reddy\OneDrive\Desktop\fs_lab> node Maths.mjs
PS C:\Users\reddy\OneDrive\Desktop\fs_lab> node index.mjs
--- greeting.mjs module ---
Hello
Goodbye
--- Maths.mjs module ---
add(10, 5) = 15
subtract(10, 5) = 5
PS C:\Users\reddy\OneDrive\Desktop\fs_lab> 
```

10.RESULT

Thus, the concept of modules in Node.js was successfully implemented and verified. A core/thirdparty module (Express) was used with `require()` to build simple servers (`app.js` and `coremodules.js`), and user-defined ES modules (`greetings.mjs` and `Math.js`) were created and their functions were successfully imported and used in a separate driver file (`index.mjs`). All test cases were executed and passed.